# oxygenation_diagnostic_mvp
# Detailed Diagnostic and Fitting-Integration Documentation

Synchronized to current module implementation

23 April 2026

## Contents


**1 Purpose** **3**

**2 Data model and entry point** **3**

2.1 Input dataclass . . . . . 3

2.2 Main API . . . . . 3

**3 Mathematical functions used by diagnostics** **3**

3.1 Helper transforms . . . . . 3

3.2 Bohr/$CO_2$/temperature correction . . . . . 4

3.3 Hill saturation for estimated arterial saturation . . . . . 4

**4 Risk-score construction** **4**

4.1 Feature-level risks . . . . . 4

4.2 Weighted composite score . . . . . 5

**5 State probabilities and class assignment** **5**

5.1 Five-state framework . . . . . 5

5.2 Score construction . . . . . 5

5.3 Confidence and certainty . . . . . 5

5.4 Alert level mapping . . . . . 6

**6 Dominant-driver explanation logic** **6**

**7 Return payload structure** **6**

**8 How this module is used in the full application** **6**

8.1 Call path . . . . . 6

8.2 Coupling to Krogh reconstruction . . . . . 7

**9 Function-to-code mapping table** **7**

**10 Output and report integration** **7**

10.1 Where diagnostic outputs are stored . . . . . 7

10.2 Export . . . . . 7

**11 Interpretation guidance for users** **8**

**12 Conclusion** **8**

# 1 Purpose

The module `oxygenation_diagnostic_mvp.py` transforms blood-gas and tissue-sensor information into a probabilistic oxygenation assessment and alert category. It is designed to be explicit and auditable: each output can be traced to explicit equations and fixed parameters.

# 2 Data model and entry point

## 2.1 Input dataclass

Main input type:

```
OxygenationInput(po2, pco2, pH, temperature_c, sensor_po2,
      hemoglobin_g_dl=13.5, venous_sat_percent=75.0)
```

Meaning of inputs:

- `po2`: blood-gas $PO_2$,
- `pco2`: blood-gas $PCO_2$,
- `pH`: blood-gas acidity,
- `temperature_c`: body temperature,
- `sensor_po2`: local tissue/sensor $PO_2$,
- `hemoglobin_g_dl`: optional hemoglobin,
- `venous_sat_percent`: optional venous saturation.

## 2.2 Main API

`alert_decision(oxygen_input, yellow_threshold=0.25, orange_threshold=0.50, red_threshold=0.75)`

Input guard: thresholds must satisfy

$$0 \leq yellow \leq orange \leq red \leq 1.$$

Otherwise a `ValueError` is raised.

# 3 Mathematical functions used by diagnostics

## 3.1 Helper transforms

- `_clamp(x)`: bounds values to a finite interval.
- `_sigmoid(x)`: $\sigma(x) = \frac{1}{1+e^{-x}}$, with internal clipping of $x \in [-60, 60]$ for numerical stability.
- `_softmax(scores)`: $p_i = \frac{e^{s_i}}{\sum_j e^{s_j}}$, with max-shift stabilization.

### 3.2 Bohr/$CO_2$/temperature correction

`effective_p50(pH, pco2, temperature_c)` computes:

$$P_{50,eff} = P_{50,std} \cdot 10^{\beta_{pH}(pH - PH_{ref}) + \beta_{CO_2} \log_{10}(pCO_2/PCO2_{ref}) + \beta_T(T - T_{ref})}.$$

Constants are imported from `krogh_app/constants.py` and match the forward model constants.

### 3.3 Hill saturation for estimated arterial saturation

`hill_saturation(po2, p50_eff)`:

$$S(P) = \frac{P^n}{P^n + P_{50,eff}^n}, \quad n = 2.7.$$

In `alert_decision`, this yields:

$$\texttt{estimated_sa_percent} = 100 \cdot S(PO_2).$$

## 4 Risk-score construction

### 4.1 Feature-level risks

Current feature transforms:

$$\begin{aligned}
R_{po2} &= \sigma\left(\frac{55 - po2}{12}\right), \\
R_{sensor} &= \sigma\left(\frac{35 - sensor_po2}{10}\right), \\
R_{acid} &= \sigma\left(\frac{7.32 - pH}{0.06}\right), \\
R_{hypercapnia} &= \sigma\left(\frac{pco2 - 48}{8}\right), \\
R_{anemia} &= \sigma\left(\frac{11.5 - Hb}{1.5}\right), \\
R_{venous} &= \sigma\left(\frac{68 - Sv}{7}\right), \\
R_{gap} &= \sigma\left(\frac{(po2 - sensor_po2) - 45}{15}\right).
\end{aligned}$$

Temperature risk is the maximum of low- and high-temperature stress sigmoids:

$$R_T = \max\left(\sigma\left(\frac{35 - T}{0.8}\right),\ \sigma\left(\frac{T - 38.5}{0.8}\right)\right).$$

Compensation-gap term used later in state scoring:

$$\texttt{compensation_gap} = \texttt{clamp}\left(\frac{po2 - sensor_po2}{80}\right).$$

## 4.2 Weighted composite score

Current weights in code:

$$R = 0.22R_{po2} + 0.24R_{sensor} + 0.08R_{gap} + 0.12R_{acid} + 0.10R_{hypercapnia} + 0.08R_{anemia} + 0.06R_{venous} + 0.02R_T.$$

Then clamp to $[0, 1]$.

Hard-floor overrides:

- if `sensor_po2 < 15` or `po2 < 25`, enforce $R \geq 0.90$,
- else if `sensor_po2 < 25` or `po2 < 35`, enforce $R \geq 0.72$.

These overrides ensure severe hypoxic signals are not diluted by averaging in the weighted score.

# 5 State probabilities and class assignment

## 5.1 Five-state framework

States in current model:

1. `normoxia`,
2. `intermediate_oxygenation`,
3. `low_oxygenation_approaching_critical`,
4. `hypoxia`,
5. `profound_hypoxia`.

## 5.2 Score construction

For each state, a Gaussian-like $PO_2$-band score is built around fixed centers and widths, then modified by:

- direct sensor $PO_2$ heuristics,
- risk-score and compensation-gap contributions,
- additional boosts for very low sensor or blood-gas $PO_2$.

Finally `_softmax` normalizes to probabilities.

## 5.3 Confidence and certainty

- `confidence`: maximum state probability,
- `certainty`: gap between highest and second-highest probability (clamped to [0,1]).

### 5.4 Alert level mapping

Using user thresholds:

- red if $R \geq red$,
- orange if $orange \leq R < red$,
- yellow if $yellow \leq R < orange$,
- green otherwise.

## 6 Dominant-driver explanation logic

`_describe_feature_risks(...)` ranks feature risks and generates:

- `dominant_risk_drivers`: top human-readable drivers,
- `driver_summary`: sentence-style summary for GUI/report text.

This is central for interpretability because the user sees not only alert color, but also the main contributors.

## 7 Return payload structure

`alert_decision(...)` returns a dictionary including:

- `predicted_state`, `risk_score`, `alert_level`,
- `confidence`, `certainty`,
- five primary probabilities,
- backward-compatible alias probabilities for existing report/GUI paths,
- `estimated_sa_percent`, `p50_eff`,
- `feature_risks`, `dominant_risk_drivers`, `driver_summary`,
- normalized `input` echo.

## 8 How this module is used in the full application

### 8.1 Call path

1. User enters diagnostic inputs in GUI diagnostic tab.
2. GUI creates `OxygenationInput` and calls diagnostic engine wrapper.
3. Wrapper calls `alert_decision(...)`.
4. GUI displays probabilistic state, risk score, certainty, alert, and driver summary.

## 8.2 Coupling to Krogh reconstruction

After diagnostic evaluation, the GUI can run inverse reconstruction:

- venous saturation is transformed to a venous $PO_2$ target,
- reconstructor fits perfusion, mitoP50, and relative metabolic demand,
- hidden hypoxic burden and radius-conditioned variants (30/50/100 µm) are reported,
- uncertainty and identifiability summaries are attached.

Therefore, diagnostics and reconstruction should be interpreted jointly.

# 9 Function-to-code mapping table

| Function | Inputs | Output role |
|---|---|---|
| `effective_p50` | pH, pCO2, temperature | Shifts oxygen affinity for current blood-gas context |
| `hill_saturation` | po2, optional p50_eff | Estimated saturation, used for `estimated_sa_percent` |
| `_state_probabilities` | risk_score, compensation_gap, sensor_po2, po2 | Five-state probability map |
| `_describe_feature_risks` | feature risk dictionary | Dominant-driver labels and narrative summary |
| `alert_decision` | `OxygenationInput`, thresholds | Main diagnostic payload for GUI/report layers |

# 10 Output and report integration

## 10.1 Where diagnostic outputs are stored

The diagnostic result dictionary is cached in GUI runtime as `last_diagnostic_result`. If reconstruction runs, `last_krogh_reconstruction` is also cached.

## 10.2 Export

Via `krogh_app/persistence.py`:

- diagnostic report export as JSON or CSV,
- publication report export as TXT/TEX/PDF,
- automatic inclusion of dominant drivers, uncertainty summary, hidden burden summary, and radius sensitivity summary.

## 11 Interpretation guidance for users

- The diagnostic score is a transparent heuristic model, not a trained clinical predictor.
- High confidence with low certainty means one state dominates only weakly over the second-best state.
- Strong alert levels should be interpreted with reconstruction assumptions (fit warnings, boundary hits, radius sensitivity).
- Repeated measurements and trend interpretation are preferred over isolated snapshots.

## 12 Conclusion

`oxygenation_diagnostic_mvp.py` provides a mathematically explicit probabilistic oxygenation interpretation layer that is tightly integrated with the mechanistic Krogh reconstruction workflow. The implementation is intentionally transparent and suitable for scientific audit and reproducible reporting.